4/3 Slides

Updates: World Models, Multiple Cars, and dealing with Collisions

World Model

Goal: Learn a world model that can imagine future racing scenes offline

- The project now has a working offline Recurrent State Space Model (RSSM) pipeline for the racing simulator.
- The purpose of the model is to learn scene dynamics from replay data and generate coherent imagined rollouts.
- The current system has moved beyond basic failure and can now be evaluated on specific weaknesses.

What Has Been Implemented

- Autoencoder sanity check for the vision stack
- Replay collection for manual and automatic world-model data
- Replay manifest preparation for train/validation splits
- RSSM training pipeline with:
  - encoder
  - GRU sequence model
  - prior and posterior latent heads
  - reward predictor
  - decoder
- Checkpoint saving
- Hallucination video generation
- Local GPU optimizations for the RTX 4070 Laptop GPU

What Is Working Right Now

- Current Hallucination Quality Is Meaningful
- Straight-line rollouts are broadly coherent
- The ego car is consistently preserved
- Grass and road corridor separation is usually preserved
- The bottom HUD-like structure is often reconstructed
- Turns are partially learned and usually move in the correct broad direction

Current Baseline Interpretation

Strong points

- Stable ego-car rendering
- Stable broad road geometry
- Coherent short-horizon scene evolution

Weak points

- Corner geometry is still patchy in some clips
- Turn consistency degrades deeper into imagined rollout
- Visual sharpness remains low, especially at longer horizons

Bottom takeaway:
Main limitation: turn consistency and long-horizon road-shape persistence.

Recent Improvements

Training and Runtime Improvements:

- Unique run-specific output directories for checkpoints and hallucinations
- Epoch-by-epoch checkpoint saving
- Epoch-by-epoch hallucination saving
- CUDA AMP enabled
- Pinned-memory and worker-based DataLoader path
- Non-blocking CUDA transfers
- Batch size increased to `16`
- Replay `window_stride` increased to `4`
- Training `sequence_length` increased to `75`

Bottom interpretation:
These changes reduced iteration cost while giving the RSSM longer temporal context for corners.

World Model Next Steps

1. Keep the current run as the baseline reference.
2. Compare several fixed validation clips instead of relying on one clip.
3. Focus evaluation on:
    - straight stability
    - turn persistence
    - road-width consistency
    - collapse rate after context ends
4. If turns remain the main weakness, collect turn-heavy manual buckets in both directions.
5. Retrain on the same architecture before changing capacity.
6. Only if turn consistency still stalls, consider increasing `rssm.stochastic_dim`.

Bottom line:

The next phase should prioritize better evaluation discipline before architecture changes.

What Success Looks Like Next for World Model

Definition of Success for the Next Iteration

- More consistent road curvature in turns
- Less abrupt change of turn direction
- Longer persistence of road edges after context ends
- More stable geometry across multiple validation clips

Multiple Cars

- Multiple Agents, Collision Reward Fixes, Latching, Blocking, and Sustained-Contact Penalties in PPO Racing

Setup: How Multi-Agent Training Works

- One environment, several cars (`num_agents > 1`)
- Torch PPO backend uses **one shared policy**
- Each car is its own stream of transitions
- Extra reward terms live in `RewardShapingWrapper` on top of env reward

Takeway:

- We do **not** train separate networks per car. We use one policy, feed it each car's observation, and each car gets its own reward and done signal. Most of the important behavior here comes from the reward wrapper, not PPO itself.

What the First Reward Stack for Multiple Cars Code Did

Tiered collision penalty based on relative speed

- low / medium / high brackets

Edge-triggered only:

- penalty applied when a pair **newly enters** collision
- tracked with `_active_collision_pairs`

Optional terms:

- rank reward
- relative velocity reward
- overtake bonus

Forward progress and time penalty drive most of the signal
Takeaway:

- Collisions were **not** modeled as “pay every frame you touch.” They were modeled as “maybe pay once when contact starts,” and the amount depended on the speed tier of the impact.

Main Problems We Observed

- Latching, Spin-Outs, and Blocking
- Cars can physically stick in Box2D when in contact
- Rear car often gets torque and spins out
- After the first contact step, the edge-triggered penalty stops charging that pair
- Policy can exploit contact to control the other car’s progress
- Cheap contact can make blocking better than clean racing

Takeaway:

- Part of the issue is physics, but part is reward design. Once cars latch, the learning signal to separate is weak, so the policy can discover blocking or glued contact as a useful strategy.

Root Cause: Why Collisions Became too Cheap

Slow bumps can fall below the lowest speed tier

- so the first collision step can get **zero** penalty

If the pair is already in `_active_collision_pairs`

- sustained contact gets **no repeat penalty**

Rank / overtaking rewards can encourage holding position

- without caring *how* position is maintained

Takeaway:

Two issues stacked together:

1. very slow contact could pay nothing on entry
2. ongoing contact was almost free after the first frame

So the system had a “doorbell” model: it rings once when contact begins, but there is no meter running while the cars stay glued together.

New Fixes Added: Two New Reward Components

- Two New Reward Components

`collision_static_penalty`

- fixed fallback when tiered collision penalty returns 0
- applies on the **first collision step** for a pair

`proximity_step_penalty`

- per-step cost for sustained close contact
- applies while a pair was colliding last step and is still close this step
- scaled by closeness

Takeaway:

- One fix handles **slow first touches that used to pay nothing**. The other handles **latched contact that used to stop costing after frame one**.

What Each New Penalty Actually Does

- One-Time Entry Cost vs. Ongoing Contact Cost

`collision_static_penalty`

- only applies on the **new-contact** branch
- replaces a zero tiered penalty with a constant
- does **not** repeat every timestep

`proximity_step_penalty`

- uses previous active collision pairs
- if still within `collision_distance_threshold`, add:
  - `proximity_step_penalty × closeness`

Closeness:

- `max(0, 1 - distance / threshold)`
- tighter contact → larger penalty

Takeaway:

- `collision_static_penalty` is the one-time patch for slow impacts.
- `proximity_step_penalty` is the sustained-contact fix — the “meter keeps running until they separate” part.

Configuration Direction and Remaining Limitations

How to Tune It:

While tuning contact, reduce or disable:

- `rank_reward_scale`
- `relative_velocity_scale`

Suggested direction:

- `collision_static_penalty`: negative, around `-2` to `-4`
- `proximity_step_penalty`: negative, around `-0.08` to `-0.15`

Optionally lower `collision_low_speed_threshold`

- so more contacts trigger at least the low tier

Configuration Direction and Remaining Limitations Continued

What Still Isn’t Solved:

Off-track termination can still be exploited

A car may learn sabotage:

- force opponent off track
- end episode
- avoid being overtaken

This is a broader game-rule + reward-design issue

Takeaway:

- These new penalties fix cheap entry contact and cheap sustained latching, but they do **not** solve every exploit. If ending the episode is better than losing position, the policy may still learn sabotage.